TrustShield AI: Hybrid ML-Based Phishing Detection using Flask, scikit-learn & MongoDB

Abstract

TrustShield AI is a multi-layered, AI-driven phishing detection framework designed to identify and mitigate sophisticated email-based attacks in real time. Built on a three-tier architecture comprising a frontend dashboard, a Flask-based asynchronous backend, and a MongoDB persistence layer, the system fuses six independent intelligence signals — machine learning prediction, URL intelligence, rule heuristics, emotional analysis, behavioural anomalies and attachment inspection — into a single sub-second verdict. Verdicts are accompanied by natural-language explanations generated by a locally hosted large language model (LLM).

The system continuously stores every analyzed email, along with its classification metadata, in a live *retraining corpus* that is later used to update the active model. This ensures that the detection engine adapts to new phishing techniques and evolving attack patterns over time. TrustShield AI integrates with email clients through a Chrome Manifest V3 browser extension and with security operations teams through a real-time SOC dashboard.

1. Overview

Phishing remains one of the most pervasive and damaging vectors in modern cybersecurity. Traditional rule-based spam filters rely on static keyword lists and sender blocklists, which attackers routinely defeat through paraphrasing, typosquatting, and personalised spear-phishing campaigns. TrustShield AI was designed to address this gap by combining classical machine learning, rule heuristics, and language-model reasoning in a single low-latency pipeline.

> "TrustShield does not replace human judgement — it protects the human inside the loop, reducing cognitive load on the people most often targeted by phishing campaigns."

The framework is intentionally lightweight. It runs on commodity hardware, uses an asynchronous Flask backend for non-blocking inference, and depends only on open-source libraries. Every architectural decision is biased toward *observability*, *explainability*, and *continuous learning*.

2. Background and motivation

Conventional email security deployments compute threat levels from static rule-sets that perform minimal processing on inbound mail. As a result, large volumes of malicious messages still reach end-users. Three structural problems motivated the design of TrustShield AI:

1. **Exposure window.** When a phishing campaign slips past static filters, the user becomes the last line of defence — a position humans are consistently poor at occupying.
2. **Rigidity of hardcoded rules.** Trigger-word filters cannot keep pace with adversarial paraphrasing or LLM-generated phishing content.
3. **Latency budget.** Robust analysis must not delay mail delivery. Any practical solution has only a few hundred milliseconds per message.

TrustShield's response is the **TrustShield Orchestrator (TSO)** — a horizontally cooperating set of intelligence modules rather than a monolithic classifier. The orchestrator coordinates analysis across three architectural tiers and six analytic layers, returning a unified verdict that is both quantitative (a trust score) and qualitative (a natural-language explanation).

3. System architecture

3.1 Three-tier design

TrustShield AI is structured into three logical tiers. This separation allows each tier to scale, fail and be replaced independently of the others.

Tier	Technology	Purpose
Presentation	SOC dashboard · Chrome extension	Surfaces verdicts, alerts and live telemetry to the user.
Application	Flask API · asyncio executor · Detection engine · LLM gateway	Performs analysis, fusion scoring and explanation generation.
Data	MongoDB · Model registry · Retrain corpus	Persists scans, model artifacts and the living dataset.

4. Technology stack

The project is implemented entirely in open-source technologies. The table below enumerates each architectural layer, the technology used, and its purpose within the system.

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript (Vanilla)	Dashboard UI, real-time updates
Backend	Flask (Python), asyncio	REST API, async processing
Database	MongoDB	Email storage, model artifacts
ML Framework	scikit-learn	Classification models
LLM	Ollama (phi model)	Natural language explanations
Browser Extension	Chrome Manifest V3	Client-side integration

Table 1. Engineering stack of TrustShield AI.

5. Detection engine

The detection engine is the analytic core of TrustShield AI. Each incoming email is normalised, vectorised and dispatched to a non-blocking executor that runs ML inference alongside five rule-driven intelligence modules. The layers do not wait on each other — the pipeline keeps moving even as the LLM generates a human-readable explanation in the background.

5.1 Aggressive fusion

TrustShield uses a strategy referred to internally as *aggressive fusion*. Every layer returns a numeric score in `[0, 1]`. A weighted base score is computed across all layers, then multiplied by a contextual **risk multiplier** when the message touches financial keywords, account-state language or high emotional intensity. A trust score below the critical threshold flips the verdict to *phishing · critical risk*.

5.2 Scoring layers

The six intelligence layers and their default fusion weights are as follows.

Layer	Signal Type	Weight	Description
ML Classifier	Machine learning prediction	0.35	scikit-learn text classification
URL Intelligence	Heuristic URL analysis	0.20	Domain reputation, TLD risk
Rule Heuristics	Pattern matching	0.15	Keyword, sender analysis
Emotional Analysis	Sentiment detection	0.10	Urgency, fear indicators
Behavioural	Anomaly detection	0.10	Unusual patterns
Attachment	File analysis	0.10	Executable detection

Table 2. Default weights of the aggressive fusion algorithm. Weights are configurable per deployment.

6. SOC dashboard

The Security Operations Centre (SOC) dashboard is a web-based interface that allows security analysts to monitor the live behaviour of TrustShield AI. It is implemented in HTML, CSS and Vanilla JavaScript, and consumes the same Flask API as the browser extension.

The dashboard surfaces four primary views:

- **Live activity feed.** Every scan and every verdict, streamed in real time.
- **Risk levels and trends.** Hourly and daily phishing pressure, segmented per tenant.
- **Model information.** Active model version, accuracy, calibration and drift indicators.
- **Alerts and notifications.** High-risk verdicts, drift alarms and pipeline failures, wired to operations channels.

7. Browser extension

The TrustShield browser extension is implemented as a Chrome *Manifest V3* extension. It reads the email body and embedded URLs the instant a message is opened inside Gmail or Outlook, posts the payload to the `/analyze` endpoint over HTTPS, and renders the verdict inline within the email client.

The extension performs four steps for each opened message:

1. Reads the email content and extracts URLs from the active DOM.
2. Sends the payload to the Flask `/analyze` endpoint with a rotating API key.
3. Displays the risk score, classification and triggered rules to the user.
4. Mirrors the verdict to the SOC dashboard via the same logging spine.

8. Living retraining dataset

A central design principle of TrustShield AI is that the model must *learn from the traffic it sees*. The system does not rely solely on static phishing corpora. Every email processed through either the dashboard or the browser extension is persisted to MongoDB, along with its classification result, confidence score, risk level, source and timestamp.

During retraining, the system retrieves this stored email data from the database and uses it to update the active machine learning model. This allows the model to adapt to **new phishing techniques and evolving attack patterns** over time. The dataset is therefore a *living*, continuously growing record of real-world email behaviour.

Dataset schema

Field	Type	Description
email_content	String	The full text of the analyzed email.
label	Enum	Classification: *phishing* or *legitimate*.
confidence_score	Float [0,1]	Model probability for the predicted class.
risk_level	Enum	Bucketed severity: low · medium · high · critical.
source	Enum	Origin of the analysis: *dashboard* or *extension*.
timestamp	ISODate	Time of analysis (UTC).

Table 3. Schema of the TrustShield retraining corpus stored in MongoDB.

This continuous-learning loop ensures that the model remains **adaptive, up-to-date and effective** against contemporary phishing campaigns, without requiring manual curation of new

training data.

9. Performance evaluation

TrustShield AI was evaluated under three representative threat scenarios. The results below were obtained on a workstation with 16 GB RAM running a single Flask worker against a MongoDB instance on the same host.

Scenario	Detection rate	Avg. latency	Notes
Standard spam	≈ 98%	< 150 ms	A cohesive filtering block formed across the corpus — almost zero leakage.
Spear-phishing	≈ 95%	< 200 ms	Manipulated domains and bespoke pretexts — fusion held the line.
Burst phishing	≥ 90%	< 220 ms	High-volume rapid-fire campaigns — the asynchronous pipeline absorbed the wave.

Table 4. Detection rates and latencies measured across three threat scenarios.

10. Future work

Several directions are currently being explored to extend the framework:

- **Transformer-based comprehension** — semantic understanding of intent rather than purely lexical features.
- **Live threat-intelligence APIs** — globally sourced telemetry as a persistent additional defence layer.
- **Edge inference** — execution of the model inside the extension itself, eliminating the network round-trip.
- **Autonomous remediation** — automatic quarantine and sender disposal triggered before the user ever sees the message.

11. See also

- Phishing
- Email filtering
- Machine learning
- Security information and event management (SIEM)
- Large language model

12. References

1. Karthikeya et al. *TrustShield AI: a multi-layer phishing detection framework*. Engineering Journal, Issue 04, 2026.
2. Pedregosa, F. et al. *Scikit-learn: Machine Learning in Python*. JMLR 12, 2011.
3. Grinberg, M. *Flask Web Development*. O'Reilly, 2018.
4. Chodorow, K. *MongoDB: The Definitive Guide*. O'Reilly, 2019.
5. Google. *Chrome Extensions: Manifest V3 documentation*. 2024.
6. Ollama Project. *phi model card*. 2024.

This blog was last edited on 26 April 2026, by Karthikeya. Text is available under the open documentation license; the source code is published on github.com/Tejus468/pfsd_project.

TrustShield Blog · Engineering Journal · 2026
Written by Karthikeya · Issue 04